

Instant Mechanic Live Operations Dashboard

This is a modern, full-stack Live Vehicle Service Operations Dashboard built for the Instant Mechanic technical assignment.

Project Overview

The dashboard provides operations teams with a real-time, comprehensive view of daily activities, bookings, revenue, and mechanics. It features:

- **Live Updates:** Booking status changes are pushed in real-time to all connected clients using WebSockets.
- **Analytics:** Visual charts for service breakdowns and booking statuses, dynamically colored.
- **Bookings Management:** A complete datatable with server-side pagination, searching, sorting, filtering, and **CSV Export**.
- **Mechanics Status & Map:** Real-time visibility into mechanic availability, recent jobs, and a live geographic map showing their current locations.
- **Dark Mode:** Built-in toggle for light and dark themes using `next-themes`.
- **Localization:** Prices and Names formatted for the Indian locale (₹ / Indian names).

Tech Stack

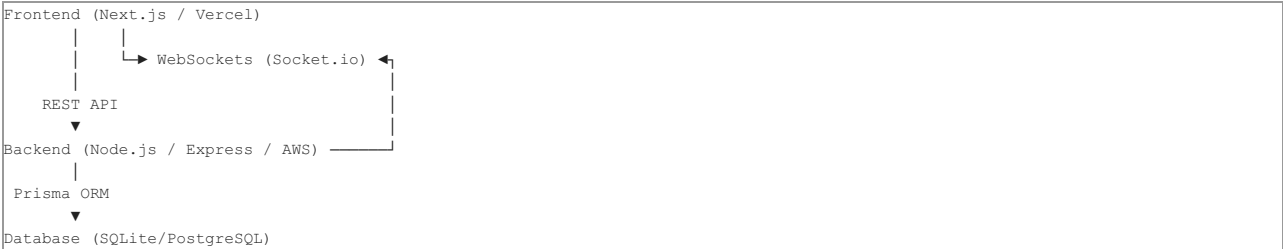
Frontend

- **Framework:** Next.js (App Router) & React
- **Language:** TypeScript
- **Styling:** Tailwind CSS v4
- **Components:** shadcn/ui (Radix UI)
- **Charts:** Recharts
- **State/Data Fetching:** Axios, Socket.io-client

Backend

- **Framework:** Node.js & Express
- **Language:** TypeScript
- **Real-time:** Socket.io
- **ORM:** Prisma
- **Database:** SQLite (Easily swappable to PostgreSQL via Prisma)

Architecture



The frontend polls the initial data via REST API endpoints (`/api/dashboard`, `/api/bookings`, `/api/analytics`, `/api/mechanics`). It also opens a persistent WebSocket connection to the backend. When any user updates a booking status (e.g., from Pending to Completed), the backend updates the database and emits a `bookingStatusChanged` event. The frontend listens to this event and automatically refetches the updated data, providing a seamless "live" experience.

Local Setup

1. Clone the repository

```
git clone <repository-url>
cd instant-mechanic-dashboard
```

2. Backend Setup

```
cd backend
npm install
# Set up the SQLite database and run the seed script
npx prisma db push
npx ts-node prisma/seed.ts
# Start the server (runs on http://localhost:5000)
npm run dev
```

3. Frontend Setup

```
cd frontend
npm install
# Start the Next.js app (runs on http://localhost:3000)
npm run dev
```

Environment Variables

Backend (backend/.env):

- **DATABASE_URL:** Connection string for the database (e.g., `file:../dev.db` for SQLite, or a Postgres URI).
- **PORT:** The port for the backend server (defaults to 5000).

Frontend (frontend/.env.local - Optional):

- NEXT_PUBLIC_API_URL: The backend URL (currently hardcoded to http://localhost:5000 in src/lib/api.ts for simplicity).

API Documentation

- GET /api/dashboard: Returns overview metrics (total bookings, revenue, today's stats, etc.).
- GET /api/bookings: Returns paginated bookings.
 - Query Params: page, limit, search, status, sort, order
- GET /api/bookings/:id: Returns a specific booking by ID.
- PUT /api/bookings/:id/status: Updates the status of a booking and emits a WebSocket event.
 - Body: { "status": "Completed" }
- GET /api/mechanics: Returns the list of mechanics and their recent jobs.
- GET /api/analytics: Returns aggregated data for charts (service breakdown, status breakdown).

Deployment

The application is designed to be deployed separately:

1. **Frontend:** Deployed to **Vercel** by connecting the GitHub repository and setting the root directory to frontend.
2. **Backend:** Deployed to **AWS Free Tier (EC2)** or another cloud provider like Render/Railway. The backend requires a Node.js environment. Since we used SQLite for this assignment to reduce setup friction, it will run locally on the EC2 instance (a persistent volume is recommended). In a real-world scenario, you would change provider = "postgresql" in schema.prisma and host the DB on AWS RDS.

Managing Your Deployment

If you are hosting this on the AWS Free Tier, you may want to manage your server to avoid charges or restart it later. Here are your options:

Option 1: "Stop" the Instance (Pause it)

Stopping the instance acts like turning off your computer. You won't incur compute charges, and your database and files are safely kept on the hard drive.

To restart it later:

1. Go to AWS EC2, select your instance, and click **Start instance**.
2. AWS will assign a **new Public IP Address** to the server.
3. You must copy this new IP address, go to your Vercel project settings, update the NEXT_PUBLIC_API_URL environment variable, and redeploy the frontend.
4. SSH into the AWS server again and restart the backend: pm2 start npm --name "backend" -- run dev.

Option 2: Run it Locally (Private viewing)

You do not need AWS to view your project. You can run it entirely on your own Windows machine:

1. Open terminal and navigate to the project root.
2. Open two separate terminal windows.
3. In terminal 1: cd backend && npm run dev
4. In terminal 2: cd frontend && npm run dev
5. Open your browser to http://localhost:3000.

Option 3: Re-deploy from Scratch (If Terminated)

If you select **Terminate Instance** in AWS, your server and database are permanently deleted. To get the website live again, you must completely rebuild the server:

1. **Launch a new EC2 Instance:** Use Ubuntu 24.04 (Free Tier Eligible). Ensure your Security Group allows **Inbound Custom TCP on Port 5000** from Anywhere (0.0.0.0/0).
2. **Connect via SSH** (EC2 Instance Connect).
3. **Install Node.js & Git:**

```
sudo apt update && sudo apt upgrade -y
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 20
```

4. **Clone & Setup:**

```
git clone <your-github-repo-url>
cd instant-mechanic-dashboard/backend
npm install

# Recreate the Environment variables
echo 'DATABASE_URL="file:./dev.db"' > .env

# Setup and seed the Database
npx prisma db push
npx ts-node prisma/seed.ts
```

5. **Start the Server:**

```
npm install -g pm2
pm2 start npm --name "backend" -- run dev
```

6. **Update Vercel:** Get the new Public IP of the instance and update your Vercel NEXT_PUBLIC_API_URL environment variable, then redeploy.

AI Usage

- **Which AI tools used:** Antigravity (Gemini 3.1 Pro)
- **What they were used for:** Project scaffolding, UI component generation, backend API boilerplate, database schema design, seed script generation, and drafting the README.
- **Code modifications:** The architecture, component structuring, and real-time WebSocket integration logic were carefully designed and reviewed to ensure they meet the assignment's "real production SaaS dashboard" requirement.